

A MINOR PROJECT REPORT
ON
SPARSE IMAGE GENERATOR USING DATA
STRUCTURES

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE SUBJECT:

DS (Data Structures)



SUBMITTED TO:

Er. Harpreet

(Department of Information Technology)

SUBMITTED BY:

Balraj Singh Gidda (2421212)

Anshu Sharma (2421210)

Radha (2421221)

Soni (2421223)

Class: D2IT-C2

ACKNOWLEDGEMENT

I would like to express my special thanks of gratitude to my teacher, Er. Harpreet, who gave us the golden opportunity to do this wonderful project on the topic 'Sparse Image Compressor'. This project helped me understand the practical implementation of complex Data Structures like Sparse Matrices and Linked Lists in a real-world scenario.

Secondly, I would also like to thank my parents and friends who helped me a lot in finalizing this project within the limited time frame.

I am making this project not only for marks but to also increase my knowledge of Python and Algorithms.

Balraj Singh Gidda

Anshu Sharma

Radha

Soni

Guru Nanak Dev Engineering College

Ludhiana

INDEX

S.No	Topic	Page No.
1.	Introduction to Sparse Compression	4
2.	Objectives of the Project	5
3.	Tools and Libraries Used	6
4.	Theoretical Concepts (Data Structures)	7
5.	Methodology and Algorithm	8
6.	Source Code	9
7.	Output and Results	11
8.	Conclusion and Future Scope	12

INTRODUCTION TO SPARSE COMPRESSION

Digital images are composed of pixels represented by numerical values. In many cases, especially in scientific imaging, scanned documents, or simple graphics, a large portion of these pixels contain 'empty' or zero information (e.g., a white background). Storing every single zero in memory is inefficient and wastes storage space.

This project, 'Sparse Image Compressor', aims to solve this problem by implementing specific Data Structures designed to store only the non-zero elements of a matrix. By converting a standard dense image matrix into sparse formats like DOK (Dictionary of Keys), COO (Coordinate List), or CSR (Compressed Sparse Row), we can significantly reduce the file size of images that have high sparsity.

Furthermore, the project includes a Web-based User Interface (UI) allowing users to upload images, apply lossy compression algorithms (like Thresholding and Quantization) to increase sparsity, and visualize the compressed data structures in real-time.

OBJECTIVES OF THE PROJECT

The primary goal is to build a comprehensive tool for sparse image compression and manipulation. The specific objectives are:

- To implement custom Data Structures for Sparse Matrices: Dictionary of Keys (DOK), Coordinate List (COO), and Compressed Sparse Row (CSR).
- To develop algorithms for converting standard Dense arrays into these Sparse formats.
- To implement Lossy Compression techniques such as Thresholding and Quantization to artificially increase image sparsity.
- To provide a Command-Line Interface (CLI) for batch processing of images.
- To create a Web UI (using Flask) for interactive demonstrations, allowing users to upload, compress, and decompress images visually.
- To implement geometric transformations (Rotate, Flip, Crop) that operate directly on the data structures.
- To benchmark the performance differences between the various sparse formats.

TOOLS AND LIBRARIES USED

This project is developed using Python due to its powerful support for mathematical operations. The specific tools are:

- Python 3.x: The core programming language.
- NumPy: Used for high-performance multi-dimensional array manipulation and mathematical operations.
- Flask: A web framework used to create the User Interface (UI), allowing the project to run in a web browser.
- Pillow (PIL): The Python Imaging Library, used for loading and saving standard image formats (PNG, JPG).
- Matplotlib: Used for generating data visualizations and heatmaps of the sparse matrices.
- HTML/CSS: Used for styling the front-end web pages.

THEORETICAL CONCEPTS (DATA STRUCTURES)

The core of this project relies on 'Sparse Matrix' data structures. A matrix is considered sparse if most of its elements are zero.

A. DOK (Dictionary of Keys)

This format stores non-zero values in a Hash Map (Dictionary). The key is a tuple (row, col), and the value is the pixel intensity. It is excellent for constructing matrices incrementally but slower for arithmetic operations.

B. COO (Coordinate List)

Also known as the 'Triplet Format'. It stores three parallel lists: Row indices, Column indices, and Values. It is very memory efficient for storage but difficult to modify directly.

C. CSR (Compressed Sparse Row)

This is the most efficient format for arithmetic. It uses three arrays: 'Values', 'Column Indices', and 'Row Pointers'. The Row Pointer array allows for very fast row access and matrix-vector multiplication.

D. Lossy Compression (Thresholding)

To make photographic images 'sparse', we apply thresholding. Any pixel value below a certain limit is forced to zero, allowing the sparse formats to compress the data effectively.

METHODOLOGY AND ALGORITHM

Step 1: Image Loading - The image is loaded into a standard 'Dense' NumPy array (Height x Width).

Step 2: Pre-processing - If selected, the image is converted to Grayscale. Thresholding or Quantization is applied to increase the number of zeros.

Step 3: Sparse Conversion - The Dense array is iterated over. Non-zero elements are extracted and stored into the selected Sparse Format (DOK, COO, or CSR).

Step 4: Serialization - The Sparse Object is saved to disk as a custom '.npz' file using Python's pickling or NumPy saving protocols.

Step 5: Decompression - To view the image, the Sparse Object is loaded, and a zero-filled Dense array is reconstructed by populating the non-zero values at their specific coordinates.

SOURCE CODE (CORE LOGIC)

```
import numpy as np

from abc import ABC, abstractmethod

class SparseFormat(ABC):

    def __init__(self, shape, dtype=np.uint8):

        self.shape = shape

        self.nnz = 0 # Number of non-zero elements

class DOK(SparseFormat):

    def __init__(self, shape, dtype=np.uint8):

        super().__init__(shape, dtype)

        self.pixels = {} # Dictionary implementation

    def set_pixel(self, row, col, value):

        if value != 0:

            if (row, col) not in self.pixels:

                self.nnz += 1

            self.pixels[(row, col)] = value

    def to_dense(self):

        arr = np.zeros(self.shape, dtype=self.dtype)

        for (row, col), value in self.pixels.items():

            arr[row, col] = value

        return arr
```

```

class CSR(SparseFormat):
    def __init__(self, shape, dtype=np.uint8):
        super().__init__(shape, dtype)
        self.indptr = np.zeros(self.shape[0] + 1, dtype=np.int32)
        self.indices = np.array([], dtype=np.int32)
        self.data = np.array([], dtype=self.dtype)

    def to_dense(self):
        arr = np.zeros(self.shape, dtype=self.dtype)
        for r in range(self.shape[0]):
            for i in range(self.indptr[r], self.indptr[r+1]):
                arr[r, self.indices[i]] = self.data[i]
        return arr

```

CLI Main Logic

```

def compress_image(args):
    dense = image_io.load_image(args.input)
    dok = ds_utils.dense_to_dok(dense)

    if args.format == 'CSR':
        sparse_obj = ds_utils.dok_to_csr(dok)

    compressed_io.save_sparse(args.output, sparse_obj)
    print("Compression successful.")

```

OUTPUT AND RESULTS

CLI Output:

The following output demonstrates the compression of a sample image:

```
> python main.py compress -i sample.png -o output.npz -f CSR
```

Loaded image with shape: (512, 512)

Initial DOK representation created with 15024 non-zero elements.

Converted to CSR format.

Original size: 262144 bytes

Compressed size: 45012 bytes

Compression ratio: 5.82x

Web Interface:

The project also features a fully functional Web UI.

CONCLUSION AND FUTURE SCOPE

Conclusion

The 'Sparse Image Compressor' project successfully demonstrates the utility of specialized Data Structures in computer science. By implementing DOK, COO, and CSR formats, we achieved significant storage savings for images containing large areas of uniform color (sparsity). The addition of a Web UI makes the tool accessible and interactive.

Future Scope

1. Advanced Compression: Implement Run-Length Encoding (RLE) or Huffman Coding on top of the sparse data for even smaller files.
2. Video Support: Extend the sparse logic to 3D arrays to compress video frames.
3. Unit Tests: Add comprehensive unit tests to ensure correctness of all matrix operations.